

TaskExecutor propio y prioridades

La última pieza: dónde corre el warm-up

Queda una pieza suelta: dónde corre el trabajo

El warm-up ya funciona de principio a fin: un evento se publica, un listener `@Async` lo recoge y recalienta la caché en segundo plano, sincronizado con el commit.

Hasta ahora corre en el pool de hilos genérico que Spring monta por defecto para cualquier `@Async` del proyecto, compartido con cualquier otra tarea asíncrona — sin que tú hayas decidido nada al respecto. Hoy cierras el tema tomando ese control.

Por qué un pool

Crear un hilo nuevo por cada tarea es como contratar y despedir a un operario distinto por cada tarea, por pequeña que sea. Cada hilo tiene un coste real de creación y ocupa memoria propia mientras existe.

Un pool de hilos: grupo reutilizable que toma tareas de una cola



Tres parámetros lo definen

- **corePoolSize:**
 - tamaño base del pool; los hilos se crean a medida que llegan tareas hasta ese número y, una vez creados, permanecen disponibles.
- **maxPoolSize:**
 - el límite máximo de hilos que el pool puede llegar a crear si la carga aumenta.
- **queueCapacity:**
 - cuántas tareas pueden esperar en cola antes de crear más hilos (hasta maxPoolSize) o, si ya está en el máximo, rechazar la tarea.

Si la cola se llena: la política de rechazo

Si la cola se llena y ya se ha alcanzado `maxPoolSize`, qué pasa con la tarea nueva lo decide el `RejectedExecutionHandler` del pool:

Política	Qué hace con la tarea que no cabe
AbortPolicy (por defecto)	Lanza <code>RejectedExecutionException</code> — quien publicó se entera del rechazo
CallerRunsPolicy	La ejecuta el propio hilo que la publicaba — frena a quien publica, sin perder la tarea
DiscardPolicy	La descarta en silencio, sin avisar a nadie
DiscardOldestPolicy	Descarta la tarea más antigua de la cola, para hacer sitio a la nueva

Ninguna es "la correcta" en abstracto — depende de si perder una tarea es aceptable o si hace falta enterarse siempre de un rechazo.

La prioridad de un hilo: qué controla de verdad

La prioridad es una pista que le das al planificador del sistema operativo — no un mecanismo nuevo, ni una reserva de CPU. En Java va de 1 a 10, con 5 (NORM_PRIORITY) por defecto.

	Prioridad de hilo (setThreadPriority)
Qué hace	Da una pista al planificador del SO sobre a quién preferir
Qué no garantiza	Que el hilo de baja prioridad se quede sin CPU, ni que el de alta la consiga siempre
Cuándo se nota	Solo si hay contención real por los mismos núcleos
Portabilidad	El mismo número puede comportarse distinto según el sistema operativo

Por qué pools acotados, y no jugar con prioridades

Linux, Windows y macOS no tratan la prioridad de Java igual, y solo se nota cuando hay contención real. Si hay núcleos libres, o los hilos pasan la mayor parte del tiempo esperando (sleep, una respuesta de BD), la prioridad casi no cambia nada observable.

Acotar `corePoolSize/maxPoolSize` es una garantía real (como mucho existirán tantos hilos de warm-up como hayas fijado, en cualquier SO), mientras que la prioridad es, como mucho, una esperanza.

Executor por defecto vs. TaskExecutor propio

	Executor por defecto	TaskExecutor propio
¿Quién lo usa?	Las tareas que usen el executor por defecto	Este listener lo selecciona explícitamente
Nombre de hilo en el log	Genérico (task-1, task-2...)	Reconocible (warmup-1, warmup-2...)
Tamaño del pool	El que decida Spring por defecto	El que tú decidas, ajustado a esta tarea

Si mañana añades otra tarea @Async (un envío de correo), competiría por los mismos hilos que el warm-up, y en el log ambas aparecerían con el mismo nombre genérico.

Un TaskExecutor propio para el warm-up

```
@Bean(name = "warmupExecutor")
public TaskExecutor warmupExecutor() {
    ThreadPoolTaskExecutor executor = new ThreadPoolTaskExecutor();
    executor.setCorePoolSize(2);
    executor.setMaxPoolSize(4);
    executor.setQueueCapacity(20);
    executor.setThreadNamePrefix("warmup-");
    executor.setThreadPriority(Thread.MIN_PRIORITY);
    executor.initialize();
    return executor;
}
```

Cada valor tiene un porqué concreto

Parámetro	Valor	Por qué
corePoolSize	2	Hasta 2 tareas simultáneas antes de esperar en cola
maxPoolSize	4	Límite bajo a propósito: un pico no debe crecer sin control
queueCapacity	20	Margen para picos breves, sin disparar maxPoolSize enseguida
threadNamePrefix	"warmup-"	La herramienta de depuración más útil: se distingue en el log de los http-nio-* o los genéricos
threadPriority	MIN_PRIORITY	Marca esta tarea de fondo como candidata a ceder el paso

Dirigir @Async a este pool concreto

```
@TransactionalEventListener(phase = TransactionPhase.AFTER_COMMIT)
@Async("warmupExecutor")
public void onTopNovedadesInvalidado(TopNovedadesInvalidadoEvent event) {
    // ...
}
```

Separarlo permite limitar cuántos warm-ups pueden ejecutarse a la vez y evita que compitan por los mismos hilos con otras tareas @Async. Las peticiones HTTP siguen teniendo el pool de Tomcat, aunque todos esos hilos sí pueden competir indirectamente por CPU o conexiones de base de datos.

Ideas clave

- Un pool de hilos se define por `corePoolSize`, `maxPoolSize` y `queueCapacity`. Si la cola se llena con el pool al máximo, la política de rechazo decide qué pasa con la tarea que no cabe.
- La prioridad de un hilo es una sugerencia al planificador del SO, no una garantía — por eso se prefieren pools separados y acotados antes que confiar en prioridades.
- `threadNamePrefix` es la herramienta de depuración más práctica: hilos identificables a simple vista en logs.
- `@Async("nombreDelBean")` dirige la ejecución a un `TaskExecutor` concreto; separar pools permite aislar tareas asíncronas con necesidades distintas.
- Documentar la configuración (por qué esos tamaños, por qué esa prioridad) es parte del trabajo de depuración y documentación, no un extra opcional.